

Donghyung Lee

DevOps Engineer | Career Description

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- **EKS / GKE production cost optimization** - 20% reduction via hybrid transition, 50% via GCP migration — re-platforming Fargate onto GKE node pools plus an MSP reseller discount
- **Solo greenfield EKS build & launch** - the AWS production environment for a new externally-published game — platform components 100% declarative via GitOps, keyless authentication via IRSA / Pod Identity, soft launch opened 2026.07
- **IaC standardization** - codified on-premises, AWS and GCP with Terraform and Ansible — 100% IaC for new components, 100% Git-traceable change history, the same pipeline rebuildable in 30 minutes
- **GPU infrastructure for AI workloads** - a two-card A100 40GB training server on GCP built as Terraform IaC (NerdyStar), and an in-house LLM inference service on an on-prem GPU box (Phantom)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- **Pipeline design & hardening** - designed multi-cluster CI/CD pipelines on GitHub Actions, GitLab CI, Jenkins and ArgoCD with the deployment automation around them
- **Deploy productivity & reliability** - 67% deployment time reduction with continuous GitOps deployment shortening time to market
- **Canary zero-downtime rollout** - moved the production game service from Blue-Green to Argo Rollouts Canary, removing the 503 race at promotion
- **CI toolchain standardization** - a layered shared toolchain image plus an automated version cycle cut CI build-environment provisioning from 5min to ~10s (~97%), with 100% tool-version consistency

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- **Declarative logging modernization** - ELK → EFK → ECK Operator-based Elastic Stack 9.x CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- **Metrics monitoring** - kube-prometheus-stack with custom alert rules and Grafana dashboards, plus multi-DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) folded into one surface
- **Tracing cost & accuracy** - wired OpenTelemetry + Tempo to complete the Metrics ↔ Traces ↔ Logs picture, splitting the sampling point to keep metric accuracy while cutting trace storage cost

Automation & Team Enablement

Operations automation tooling with Python and Bash, plus training and standardized materials that let client ops teams run things themselves

- **Ops toolchain automation** - migrated shell → Python (stdlib only) and built the wave-sequenced auto-deploy plus auto-upgrade MR pipeline
- **Engineering enablement** - designed and ran a 6-month hands-on Kubernetes program for the DP (Delivery Partner) ops team (avg 10 attendees, 4h/session), with the materials standardized for reuse
- **Open engineering, continued** - a pipeline that collects, quality-checks and republishes financial and economic data from 10+ public APIs, running twice daily on weekdays ([economy dashboard](#)), alongside a backend-free [diagram editor](#) run in the open

Kubernetes Platform Modernization & Open-Source Contribution

Network/logging stack migration, public charts released as open source, and upstream contribution

- **Gateway API migration** - migrated multiple ingress-nginx instances onto a single NGINX Gateway Fabric control-plane (incident-free full cutover, wildcard TLS consolidated)
- **Open-source shared assets** - released a number of open-source Helm charts alongside Kubernetes CRD controllers, composite GitHub Actions and Ansible collections on ArtifactHub / GitHub Marketplace / Ansible Galaxy
- **CNCF upstream contribution** - Gateway API HTTPRoute support to Helm charts in apache/airflow, OpenTelemetry and prometheus-community, plus bug fixes to Go projects

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator SSO and Vaultwarden

- **Central authentication (SSO/OIDC)** - Keycloak Operator brought single sign-on onto existing GitLab accounts, with ArgoCD / Harbor / Vaultwarden OIDC integrated and verified
- **In-house secret governance** - deployed Vaultwarden on Kubernetes via Helm with GitLab SSO, automated backups and an interactive restore script — central management and a recovery path
- **GitOps secrets** - replaced plaintext DB passwords in Git with External Secrets Operator wired to AWS Secrets Manager, standardizing the Harbor pull secret on Sealed Secrets

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build and operate a hybrid architecture — dev and qa on-premises, review and prod on AWS — that cut monthly infrastructure cost by 20%.

For the live-service game I provide second-line technical support and operations with the external publisher, and I am building on-premises/AWS infrastructure to raise development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution: hybrid architecture design & build 100% (solo) / second-line technical support and operations for the live game in collaboration with the external publisher's ops team 2024.10 ~ Present (In Operation)

Problem

The entire infrastructure ran in a single AWS environment at a high monthly cloud cost.

Non-production environments, whose traffic barely varies, were holding cloud resources around the clock — the estate needed both cost optimization and environment separation.

Approach

- **Hybrid architecture design** - analyzed cost and usage patterns per workload, then moved dev·qa on-premises and kept review·prod, the environments that need scale and high availability, on AWS
- **Network and pipeline isolation** - separated traffic at the application layer rather than standing up a site-to-site VPN, and split the image registry (dev·qa = Harbor, review·prod = ECR) so each environment builds and operates independently
- **IaC-based operational standardization** - introduced Terraform and Ansible across both on-premises and AWS, reaching 100% IaC for new components and establishing one configuration-management standard across heterogeneous infrastructure
- **IAM credential governance** - only service and CI accounts are managed as code, with console accounts kept separate. Terraform never generates user access keys — they are issued and rotated individually — so no long-lived credential is left in plaintext in the Terraform state file

Troubleshooting

- **Version-aware backend routing for game clients** - a single ALB Ingress on EKS had to route old-client and new-client requests to different backends. Combined ALB conditional routing rules on a custom HTTP header with target-group mapping, so the split needed no additional infrastructure
- **Stopped L7 connections dropping during rolling deploys** - the ALB target deregistration delay and SIGTERM timing were misaligned, so new requests reached a terminating Pod while in-flight requests were cut. A preStop hook fails the health check first, blocking new arrivals while the remaining traffic drains within the deregistration delay; RollingUpdate was tuned add-before-remove to hold capacity through the rollout — zero connection drops. Verified on a CloudWatch-backed Grafana dashboard reading ELB 5xx and Target 5xx separately across each deploy: zero 5xx, p95 held

Results

- Cut monthly infrastructure cost 20% by re-placing workloads on the analysis, with the saving reinvested into new project infrastructure
- Physically isolated dev·QA from production, containing the blast radius of any failure within one environment and securing operational stability

Project 2: Game Application CI/CD Pipeline & Shared Toolchain Standardization (App GitOps)

Contribution 100% (Sole design and build across the pipeline and CI standardization) 2024.12 ~ Present (In Operation)

Automated the game application's manual build-and-deploy into a GitOps pipeline on GitLab CI and ArgoCD, then standardized the CI toolchain image layers and version governance shared by every in-house repository on top of it, bringing build-environment provisioning and tool-version management into a single flow.

2-1. Game Application Deployment CI/CD Pipeline (App GitOps, 2024.12 ~ Present)

Problem

Developers built and deployed to the servers by hand — about 30 minutes per deploy, with human error causing frequent incidents.

Approach

- **GitOps continuous delivery pipeline** - GitLab CI wired to ArgoCD so a Git push drives build → test → deploy end to end, with ArgoCD detecting any mismatch between the configuration declared in Git and the cluster state and self-healing back to it; the deployment logic for all four environments — dev, qa, review and prod — was unified into that single pipeline, so they are managed consistently
- **Least-privilege image build environment** - to avoid granting the host administrator privilege (privileged mode) that Docker-in-Docker (DinD) requires, image builds moved to Kaniko, which builds container images in user space without a Docker daemon — closing off the CI runner's security exposure and the risk of privilege takeover at the source

Troubleshooting

- **Preventing ArgoCD cascading prune** - to stop a Git commit error from cascading into the deletion of every child Application, workload Applications are reconciled automatically with `prune: true` + `selfHeal: true`, while the top-level app-of-apps root and the AppProject layer keep `prune: false`, so resource deletion is gated behind manual verification. Every sync event is tracked in real time through Slack notifications
- **Removing static SSH keys from the data deployment path** - the CI deploy account held an SSH key at all times and connected straight to the target instances (scp/rsync); that path was replaced by uploading artifacts to an S3 transit bucket and having each instance pull them itself via SSM SendCommand — static SSH keys are eliminated from the deployment environment, closing off the credential leakage risk

Results

- 67% shorter deployment lead time (30min → 10min) — the manual build, deploy and configuration steps were unified into the pipeline, eliminating human error as a root cause of deployment incidents
- ~78% shorter source fetch time in the data deployment (~139s → 30s) — a Git optimization that fetches only the latest commit and only the files actually needed (shallow + blobless fetch) removed the transfer bottleneck
- GitOps-based continuous delivery established as routine — a steady ~100 deploys a week per core service repository, and a shorter time for a new feature to reach production
- Built a multi-repository MasterData pipeline — a design-data change is propagated to the downstream services automatically and in one batch, with runaway repeated execution of the pipeline structurally prevented

2-2. Shared Toolchain Image Layers & Automated Version Cycle (2025.09 ~ Present)

Problem

Every CI job in the individual repositories re-installed the required tools (helm · helmfile · kubectl · crane · aws-cli) on an Alpine base, so build-environment provisioning averaged 5 minutes.

Tool versions were also fragmented per repository, so adopting a new version meant editing all of them, and the reliance on the `:latest` tag left build reproducibility and vulnerability tracking hard to establish.

Approach

- **Layer 1 (Mirror)**: mirrored the tool images from the external registries (Docker Hub, GHCR) into the internal Harbor with `crane copy`, multi-arch manifests preserved, removing the external dependency
- **Layer 2 (Custom)**: built shared images on top of Layer 1 with the tools already installed (helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools), so an individual repository has the tools immediately by pinning an image tag
- **Version governance (SSOT)**: defined a shared build template (`.toolchain_build_custom`) and central version variables (`TOOLCHAIN_*_TAG`), so a downstream repository no longer tracks individual tool tags
- **Shared CI templates modularized**: the CI jobs each repository used to copy were consolidated into a shared template repository referenced through `include`.
The shared anchors (git · ssh · rules · job-config) and the main jobs (keyless AWS auth, Kaniko build, ArgoCD deploy, backup) are managed in one place, eliminating configuration drift at the source
- **Automated version cycle**: everything from upstream version detection to downstream propagation is automated, with human verification gates (human-in-the-loop) placed at the pipeline run and the Tier 2 MR approval:
 - (1) Upstream version detection (weekly scheduler)
 - (2) Layer 1 mirror auto-MR
 - (3) Layer 2 rebuild and central version-variable sync auto-MR
 - (4) Batch auto-propagation to the consumer repos (excluding the template provider repo)

Troubleshooting

- **Toolchain build failure from a cross-tool version conflict**: during an automatic version bump, helmfile 1.5.1's minimum requirement (helm 3.18.6 or later) no longer matched the pinned version (helm 3.16.4), breaking the `build_custom` build pipeline
- **Cause and resolution**: cross-tool dependencies surface only in the upstream release notes and cannot be detected automatically in advance, so a minor-guard on helm · helmfile · kubectl now bumps only patch versions within the current minor automatically, while minor and major changes are promoted by hand after a compatibility check (a planned one-off override is available through the `*_TARGET_MINOR` environment variable)

Results

- **CI provisioning time cut ~97% (5min → ~10s)** — the pre-built standard images removed the build delay, with 100% tool-version consistency across the internal repositories
- A 4-tier version governance — the scattered version points, from the build argument (ARG) through central variable management and consumer indirection to lint-based drift detection, are ordered into a single flow
- A risk-based sequential upgrade pipeline in routine use — per-component risk tiers (T1 weekly / T2 biweekly / T3 monthly + pre-flight) drive version detection, Slack alerts and wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback
- Automated new-version propagation for helm / helmfile / kubectl / crane / rclone, with the high-blast-radius image rebuild gated behind human review rather than the schedule
- Multi-architecture support — a build path for amd64 · arm64 multi-arch images via `docker buildx`

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

In a log-scattered on-premises environment, advanced centralized logging through three stages, hardened data integrity and high availability (HA) across the log collection layer, then extended the accumulated EFK pipeline into a game user-retention analytics platform.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were physically scattered across the on-premises environment, so a pod's ephemeral logs were lost whenever it was recreated; and with no unified observability in place, diagnosing and recovering from an incident (MTTR) took a long time.

Approach

- **A lighter collection and processing tier (ELK → EFK):** per-node Filebeat collection into a central Logstash tier had grown memory-hungry on the JVM-based Logstash, so collection moved to a lightweight Fluent-bit DaemonSet and parsing to Fluentd, splitting the two roles
- **Declarative cluster management (ECK Operator):** Elastic's official Helm chart had stalled for over two years, so the ECK Operator was introduced and lifecycle management moved onto declarative CRDs
- **Extension to the cloud (EKS) with keyless automation:** the on-premises stack was templated for the AWS EKS environment and deployed there, with IRSA applied to index lifecycle management (ILM) and the S3 snapshot backup path so neither needs a static credential

Troubleshooting

- **Recovery blocked by the ECK admission webhook's downgrade refusal:** an automatic version bump on the on-premises Elastic Stack referenced an image that was listed in the release feed but never published to the registry, producing `ImagePullBackOff`. An immediate rollback was attempted, but the ECK admission webhook refuses a downgrade, so the recovery path was cut off and the failure spread to the operator
- **Cause and improvement:** the core operational risk was defined as the inability to roll back rather than the deployment failure itself. The upgrade pipeline was systematized with a registry existence check for the image, a cluster health check, a snapshot-backup warning on a major version bump, and a safe rollback procedure that accounts for the webhook constraint, preventing recurrence

Results

- Central aggregation for logs from distributed nodes and ephemeral pods — log loss when a pod is recreated or terminated is eliminated at the source, and time-series log tracing in a single Kibana view shortened the time to pin down the cause of an incident
- Operations automated and the Elastic Stack major version brought current — the manual work of TLS certificate renewal, account provisioning and node replacement is automated (declaring the version in the CRD is enough for a rolling upgrade), and the Elastic Stack major upgrade stalled for over two years was completed (8.x → 9.x)

3-2. Fluent-bit Log Collector Data Integrity & HA Hardening (2026.03 ~ Present)

Data integrity through a persisted offset: the SQLite offset DB is kept on node-local storage (`hostPath`) so the read position survives a collector pod restart (zero re-index, zero duplicate collection achieved)

Stall detection: monitoring and alerts are wired to detect the abnormal stall state in which log collection halts without the process going down, preventing the log loss that would otherwise go unnoticed

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

The EFK pipeline extended into a product-analytics platform: the search-and-incident log infrastructure was extended so an Elasticsearch Transform (every 5 minutes) pivots in-game events to one row per user

Retention metrics on a dashboard: D+1~D+30 retention and cohort analysis were built on the existing EFK pipeline and Kibana alone, with no separate commercial SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server · Self-Service Ops Console)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 6 internal tools to automate repetitive manual tasks and boost development team productivity, finally consolidating them into a self-service console platform behind a single authentication gate.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Deployed a Python bot on Kubernetes auto-sending completion messages and download QR codes to Slack on Jenkins APK build finish — 90% QA delivery-time reduction, version confusion eliminated

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

One-way sync of design documents from GitLab into a Google Drive shared drive via rclone, triggered on push — comparing on `--checksum` so only changed files transfer, with a sparse partial clone of just the changed directory cutting how much of the large repository moves — 80% doc-management reduction (10min→2min), GitLab held as the single source of truth with direct Drive edits overwritten on the next sync

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Ran an in-house LLM on an on-prem GPU server with Ollama and Open WebUI on docker compose (used by the dev team)

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ Present)

Built and open-sourced git-bridge, a Go mirroring tool, to resolve the gaps and lag of manual syncing between CodeCommit, GitLab and GitHub — automated the syncing between repositories, with failure retry and Slack notifications making sync status trackable in real time. Then placed a confirmation guard in front of data-loss operations such as force-overwrite and delete, and built a sync-history console so what synced, when and how can be traced after the fact — the history window widened 10× (up to 5,000 events) for long-range observability, and a graceful-shutdown drain that waits for in-flight syncs to finish (measured median 2.9s) stops a deploy from losing a sync.

In house it runs as the standing sync path between the two GitLab instances — the new one that holds the source and the old one that serves backup and external exposure

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

To replace an existing file server that had stopped being maintained and lacked the core features — directory UI, search, ZIP batch download — built a Go static file server in-house and released it as open source, deployed to the cluster with an in-house Helm chart wired to NFS storage, and it now carries the APK release pipeline's build artifacts as a full replacement for the previous tool

4-6. Internal Self-Service Operations Console Platform (2026.07 ~ Present)

Problem

Repetitive operational work such as creating and deleting accounts depended on shell scripts only the infrastructure engineer could run, so every request passed through one person.

The 10 internal account systems were also fragmented, leaving no way to track or audit which accounts and permissions a given person still held.

Approach

- **A single OIDC authentication gateway:** authentication no longer has to be implemented inside each application — a central OIDC gateway in front of the portal admits only authenticated traffic to the backends, and adding a service is a single configuration entry
- **Internal tools and operations services behind one portal:** the existing internal web tools plus 5 new operations services (accounts · Keycloak · Harbor · Elasticsearch · VPN) — 7 applications in all — are served through a single entry point
- **Ten account systems wired in, with an audit trail:** the provisioning APIs of the 10 scattered account systems are wired into the console for unified query and control, and every account change — create, update, delete — is recorded together with who performed it

Troubleshooting

- **Downtime from a race between the CI build and the GitOps sync:** the manifest is committed before the container image has finished building, so ArgoCD synced an image tag the registry did not yet have and the pods fell into `ImagePullBackOff`. Six of the eight services use RWO volumes and therefore the `Recreate` strategy, which terminates the existing pod first — so that window showed up as a full outage
- **Cause and improvement:** an ArgoCD PreSync hook now actually pulls the target image tag before the sync phase, so ArgoCD leaves the existing Deployment untouched until the image has landed — removing the full outage caused by a missing image

Results

- **The operational bottleneck removed through self-service** – designed and built solo over 6 weeks, so the account and permission requests that had to pass through one engineer are now handled by each team directly from the console
 - **One entry point and a central governance view** – the 7 internal tools are consolidated behind a single OIDC authentication gate, and the state of the 10 account systems can be queried and audited per person on one screen
-

Project 5: Kubernetes Cluster & Internal Systems Operations Enhancement

Contribution 100% (Solo design, with the standard procedures captured as automation scripts the team reuses) 2025.12 ~ Present

To keep the cluster running reliably, overhauled monitoring, automated Kubernetes upgrades and Helm management, moved the network controller onto the Gateway API, and went on to resolve storage bottlenecks and modernize the deploy/upgrade pipeline.

Beyond the cluster, backup and recovery across the internal systems and the long-stalled major upgrade of the in-house GitLab followed.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack and integrated DB, Node Exporter and Cilium metrics. Alert rules are broken down by component group and by severity, and an inhibit rule suppresses the lower alerts while a higher one is firing, so one incident is never notified twice.

Prometheus itself later went down when its CRD was deleted from outside the cluster and nobody noticed — which surfaced the structural limit that a watcher inside the same failure domain as the thing it watches fails with it. A cron watchdog on a bastion outside the cluster now actively checks every 5 minutes that the monitoring stack is alive (an in-cluster CronJob was rejected, since it shares the fate of what it is meant to watch).

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Scripted the Kubespray upgrade procedure and the nine mandatory post-upgrade checks (nodes, etcd, API server, certificate expiry and more), and standardized version management with a Helm framework and canonical-template synchronization.

Verification time fell 90%, and scripting certificate renewal removed expiry outages at the source.

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Consolidated the on-premises ingress-nginx instances onto a single NGF 2.x control plane with per-class Gateway CRs.

MetalLB IPs were preserved so the cutover needed no DNS or firewall change and completed without incident (30-60s per class), and one wildcard certificate cut the manual renewal effort 50%.

5-4. Storage Performance Optimization: etcd / DB SSD Migration and Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Diagnosed the bottleneck in which GitLab Runner (buildx) disk-I/O spikes drove second-scale latency on HDD-backed etcd and the game DB, migrated the control plane and the DB worker to SSD, and at the same time fully isolated the build workload onto a dedicated VM on separate physical hardware.

Game DB COMMIT latency was normalized from 9~14s to the millisecond range, with zero incidents of blocked in-game connections or service latency during builds.

5-5. Kubernetes Infrastructure Deployment Automation & Multi-cluster GitOps (ArgoCD) Migration (2026.03 ~ Present)

Problem

The Shell/awk-based infrastructure monorepo could not be test-automated — complex YAML manipulation and per-environment branching had outgrown it — and CI-runner-centred push deployment (`helmfile apply`) could not detect cluster state drift and did not scale to multiple clusters.

Approach

- **Tooling refactored onto the Python standard library:** Python3 modules that take no third-party package, with a 1:1 module-to-test `unittest` suite, giving identical execution on a local machine and on a CI runner
- **Wave-based deployment and upgrade automation:** a six-stage wave order derived from component dependencies, plus a pipeline that detects a new chart version and opens an MR carrying the values diff and the compatibility checks
- **Moved onto an ArgoCD pull-GitOps control plane:** ApplicationSet (Matrix / Git generators) automates deployment across the on-prem and AWS clusters, and existing Helm releases were adopted into ArgoCD management without recreating a single resource and without an interruption
- **GitOps governance and safety rails:** cascading prune blocked at the top layer plus an AppProject whitelist, isolating the blast radius

Results

- Platform releases across the on-prem and AWS clusters are controlled declaratively from a single GitOps control plane
- The legacy shell scripts were migrated wholesale to Python tooling backed by unit tests, making deployment dependable
- Automated chart-upgrade MRs and wave deployment together cut the operational overhead of running the infrastructure sharply

- Declarative self-healing of the desired state and per-layer prune control eliminated the risk of a large-scale deletion incident at the source

5-6. Backup & Disaster-Recovery with Restore Verification (2026.08 ~ 2026.09)

Built backup and recovery across 8 internal systems and 2 NAS paths, and set the verification standard at 'can it actually be restored' rather than 'did the backup run' — which surfaced and fixed a latent defect where a backup job exited 0 even when the database dump was truncated, so the restore would have failed.

Moving the test from 'the script completed' to 'the restored data is consistent' is what makes the disaster recovery trustworthy in practice

5-7. Splitting the In-House GitLab in Two, and the Legacy Major Upgrade: a 21-Hop Path (new instance 2024.10 / upgrade 6 days, 2026.08 ~ 2026.09)

Right after joining, a new bare-metal GitLab was built and made the source repository for new projects, with the existing repositories that were still needed moved across by GitLab mirroring. The old GitLab (containerized) was kept for backup and external exposure, and the two instances are kept in sync by git-bridge, the tool I wrote.

The old instance, which had stalled, was later brought up the official upgrade path — 21 hops over 6 days. Versions were not skipped and the path was walked as published, because skipping the migrations an intermediate version performs is what removes the way back. Downtime on the first hop was about 6.5 minutes (including a one-off volume move), and after the upgrade the nightly DR replica went 123G → 39G (−68%).

Project 6: Infrastructure Security & Governance

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Standardized the fragmented account authentication, secret management and remote-access paths into one internal security governance model.

6-1. Central Authentication (SSO/OIDC) on Keycloak Operator (2026.04 ~ Present)

Consolidated the GitLab OIDC paths — wired separately into each in-house tool (Harbor, ArgoCD, Vaultwarden) — onto a central IdP on the Keycloak Operator.

The permission flow GitLab group → Keycloak group mapper → ArgoCD role now runs through one place, giving central control of the authentication policy and a single sign-on for every tool

6-2. Secret Governance: Vaultwarden · GitOps Secrets (2026.04 ~ Present)

Moved the shared internal passwords scattered across mail and messengers onto Vaultwarden on Kubernetes, wired to SSO, establishing one place to manage them.

DB credentials that had been committed to Git in plaintext moved onto External Secrets Operator backed by AWS Secrets Manager, and the Harbor cluster secret was standardized on Sealed Secrets — closing off credential exposure in the GitOps repository at the source

6-3. Secure Remote Access to the Internal Network (OpenVPN PKI, 2026.07)

Built an on-premises VPN gateway for the internal network sized for 50 concurrent connections on a PKI, with per-user certificate issuing and revocation automated end to end by idempotent shell scripts

Project 7: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (solo design, build & operation) 2026.06 ~ Present (Building & Operating)

Solely built a greenfield EKS production environment for the commercial launch of a new externally-published game (7-1), then hardened it with distributed tracing (7-2) and Canary zero-downtime delivery (7-3) through to the soft launch (7-4), adding SLI/SLO with error budgets (7-5) and edge security for the game CDN (7-6) once live traffic was there to measure.

7-1. Greenfield EKS Production Infrastructure & Live-traffic Tuning (2026.06 ~ Present)

Problem

A single on-premises cluster had reached its limits for global traffic and flexible autoscaling, so an independent, highly available EKS production architecture was needed for the overseas commercial launch.

Approach

- **Hybrid multi-cluster, declaratively managed:** stood up a greenfield EKS cluster and brought every platform component under one declarative GitOps (ArgoCD) model
- **Compute separated and tuned on Karpenter:** the game services sit on on-demand nodes for stability, while CI runners spread across a diversified ARM multi-family spot pool, cutting cost and minimising reclaim

- **Networking and authentication automated:** AWS Load Balancer Controller, ExternalDNS and a wildcard ACM certificate together automate ingress and TLS issuance end to end, and every ServiceAccount runs on Pod Identity / IRSA, removing static IAM keys
- **Logging and secret foundation:** an eck-operator-based HA EFK stack (3 nodes across 3 AZs, EBS gp3, S3 snapshot SLM) plus External Secrets Operator gave the new cluster its own log collection and secret-reference path
- **Credentials isolated out of the deploy path:** client artifacts ship Jenkins → S3 → CloudFront, and server manifests reach the bastion's EFS mount through an S3 transit bucket and SSM SendCommand, so no SSH key is held anywhere in the path

Troubleshooting

- **Node scale-down and live game sessions reconciled:** blocking reclaim outright (`karpenter.sh/do-not-disrupt`) would have left idle nodes billing, so it was ruled out; with sessions externalised to Redis the services are stateless, and preStop drain + PDB (`minAvailable: 1`) + a disruption budget of `nodes: 1` retire one node at a time safely
- **Idle nodes never reclaimed after a rolling deploy:** diagnosed an over-sized memory request (2 GiB against a measured 101 MiB) as the cause of lopsided node occupancy (92% against 35%) and of consolidation never firing. Bringing the request down to 512 MiB and adjusting the consolidation condition restored bin-packing density and automatic reclaim
- **Traffic reaching a pod that was still starting:** intermittent 5xx errors from the ALB forwarding to a pod mid-startup were resolved with an ALB pod readiness gate — a pod counts as ready only once it has passed the target-group health check

Results

- An independent AWS hybrid multi-cluster for the overseas launch, with 100% of the platform manifests under declarative GitOps management
- Static credentials removed across the infrastructure, the bastion SSH key included
- Production and review workloads share one NodePool, removing the minimum one node each separate pool would have held and maximising bin-packing density
- The GitOps manifests and the CI/CD migration procedure are documented as a standard runbook, so the work can be handed over and the same environment rebuilt

7-2. OpenTelemetry End-to-end Distributed Tracing Pipeline (OTel + Tempo, 2026.06 ~ Present)

Problem

The existing observability stack — EFK for logs, Prometheus for metrics — gave only a limited view of how calls flow between microservices, where the latency bottleneck sits, and how an error propagates across a distributed environment.

Approach

- **Tracing stack with non-invasive instrumentation:** an OpenTelemetry Collector and Grafana Tempo (S3 backend, wired through EKS Pod Identity with no static key) on AWS EKS, with the OTel Operator's auto-instrumentation injecting the per-language SDK without touching application code
- **Sampling split for observability accuracy and storage cost:** RED metrics are derived from 100% of spans before sampling, keeping the aggregates exact, while storage goes through tail sampling — every error, every request over 500ms, and 10% of normal requests — to cut storage cost

Results

- **3-signal cross-linking:** metrics, traces and logs are linked so one Grafana view moves between them, making the call flow between services and its bottlenecks visible (the existing logging is Project 3, metrics 5-1)
- **Two-layer routing to fix trace fragmentation:** identified a structural flaw in the multi-replica collector — each app pod's gRPC connection is pinned to a different instance, so a trace that crosses pod boundaries can have its spans scattered, which can break both keep-every-error-trace and the service-graph join
- Split the pipeline into a receiving layer (layer 1) and a sampling layer (layer 2), routing by trace ID through the loadbalancing exporter over a headless Service, so every span of a trace always reaches the same sampling instance
- Validated on live traffic: the per-replica share, skewed by connection pinning to 1,650 : 541 in series count, evened out after trace-ID hashing to 4.09 : 5.18 spans/s
- **Closing the sampling-layer alert gap:** after the split, the existing collector-down alert could not see a full outage of the sampling layer; four alerts were added — sampling layer down, traces dropped undecided on buffer overflow, buffer near full, and policy evaluation errors — to watch for silent trace loss

7-3. Zero-downtime Delivery on Argo Rollouts (Blue-Green → Canary, 2026.06 ~ Present)

Problem

Blue-Green promotion swapped the ALB target group wholesale, leaving a window in which the healthy target count was 0 — and the ALB answered 503 intermittently during it.

Approach

- **Moved to a canary rollout:** within the same ALB target group, `maxUnavailable: 0` plus add-before-remove (the new pod registers before the old one is withdrawn) lets old and new coexist briefly, removing the 503 race
- **Resting on a stateless design:** the game services are stateless HTTP (state externalised to JWT and Redis/TypeORM), so traffic hitting both old and new mid-rollout is safe.

A change that cannot have both versions live at once — an API contract change, say — falls outside this strategy and is handled in a scheduled maintenance window

- **Entry and exit lifecycles synchronised:** an ALB pod readiness gate admits a pod to the traffic pool only once it can actually serve, and on the way out a deregistration delay of 30s → preStop 35s → terminationGracePeriodSeconds 60s drains in order, so in-flight requests finish — 75s for the .NET service, which drains its own in-flight requests on SIGTERM
- **Strategy templated, controller made HA:** the delivery strategy is a template in the shared Helm chart and canary is applied to the one public-facing production game service (the same game's internal services, which take no public traffic, stay ordinary Deployments), while the controller runs leader-election HA behind a PDB so a rollout survives node churn

Results

- The healthy-target-0 window and the 503 race that came with promoting a production game deploy are gone at the source; a backward-compatible patch now ships with no downtime

7-4. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft launch went live 2026-07-14 (KST) — the limited release's small-scale real traffic handled on the Canary deploys and 3-signal observability built above
- Observability, deployment and autoscaling (HPA · Karpenter) all in place for the 2026 Q4 global launch (no scale-out has fired yet at soft-launch volume)
- Re-sized Pod requests and limits from two weeks of observed soft-launch traffic (445 RPS peak, zero OOMKills) — fixed the set-up in which over-sized requests kept the HPA from ever triggering a scale-out, and cut CPU 48% (2,200m → 1,150m) and memory 71% (6GiB → 1.75GiB)

7-5. SLI/SLO and Error Budgets (2026.08)

Problem

Following the soft launch, live traffic was being served, but there was no quantitative consensus on what constituted a healthy service state. Without baseline metrics, incident triage relied on subjective judgment, leaving no clear criteria to prioritize engineering efforts for system stability.

Approach

- **Adopting Only Verifiably Measured Signals as SLIs:** Rather than enforcing theoretical metrics, validated incoming telemetry to confirm practical computational feasibility before recording SLIs — SLO targets and error budgets are selectable on the dashboard for comparison, with the committed target deliberately deferred until a baseline has accumulated
- **Deliberately Deferring Distorted Availability SLIs:** Identified an instrumentation issue where the game server returned HTTP 200 on exceptions (resulting in 0 ERROR spans out of ~7.9M). Since an availability SLI based on this would falsely read 100% and fail to burn error budget during outages, it was intentionally deferred until application telemetry was fixed.

Results

- Recorded only the latency signals that passed live-traffic validation as SLIs, and kept the distorted availability metric off the dashboard, eliminating the risk of misreading an incident as healthy
- Isolated the blocker for the availability SLI to application-level exception handling (returning HTTP 200 on errors), providing quantifiable trace data and actionable remediation requirements to development teams.

7-6. Edge Security for the Game CDN: AWS WAFv2 (2026.07)

Approach

- **Proactive False-Positive Prevention via Count Mode:** Deployed four AWS Managed Rule (AMR) groups in Count mode to observe rule by rule what live traffic they match, and designed the path from false-positive review → per-rule exceptions → Block as a single tfvars flip — eliminating the risk of legitimate-user disruption and support escalations from unverified blocking
- **Contained WCU Capacity and Predictable Costs:** Kept rule capacity within limits at 1,125 of the 1,500 WCU ceiling, calculating fixed operating costs at roughly \$9/month to ensure infrastructure budget predictability
- **Deliberate Deferral of Rate-Based Rules:** Postponed rate limiting due to high false-positive risks under mobile CGNAT (shared carrier IPs) and low observed peaks (28 reqs / 5 min) — documented clear scale thresholds for future adoption rather than enforcing premature restrictions

Results

- Introduced AWS WAFv2 in front of the game CDN with zero impact on legitimate players — four managed rule groups are observing live traffic in Count mode, with the conditions for switching to Block and for enabling the deferred rate-based rule codified alongside their technical rationale

Project 8: Second EKS Cluster on Auto Mode & a Site-to-Site VPN (WireGuard) Gateway

Contribution 100% (solo design & build) 2026.08 ~ Present (Building & Migrating)

Problem

The new publishing title had to reach the publisher's internal authentication account, and that path could not cross the public internet — it needed a private route over VPC peering. The existing production VPC's address range did not meet the peering requirement, which made recreating the VPC and everything above it, the cluster and the data tier included, unavoidable; and the publisher asked for the rebuilt cluster to run on EKS Auto Mode.

The existing cluster ran self-operated Karpenter on managed node groups, so the compute-side Terraform and state management points had grown large and node lifecycle, spot interruption and the core addons were all managed by hand. VPC peering is also non-transitive, so a private path from on-premises to the internal (private) ALB had to be provided separately.

Approach

- **Compute layer moved onto EKS Auto Mode:** node provisioning and lifecycle, the supporting AWS resources Karpenter needs (IAM, SQS, EventBridge) and the built-in addons (LB controller, EBS CSI) all moved to AWS-managed, leaving only a declarative NodePool and sharply reducing the complexity of the infrastructure code
- **Re-platformed around the IPAM constraint:** with no room to grow the subnets (four /20s exhaust the parent /18 exactly), isolation moved from physically separate subnets to security groups inside a single private subnet, relocating Aurora, Valkey and EFS
- **Site-to-site WireGuard VPN gateway:** adding another public ALB would have split one service across a second hostname, certificate and DNS record, multiplying again with every new environment, so a tunnel was chosen instead — one domain and one certificate kept intact while all internal traffic stays on a private path.
The tunnel is established outbound from on-premises toward AWS (EIP), with the peer endpoint deliberately left empty so the far-side address is learned from the first handshake
- **Platform components ported for compatibility:** 16 core platform components — observability, security, the CI toolchain — were reworked for Auto Mode, and 10 of them moved onto the ArgoCD pull-GitOps track

Troubleshooting

- **Bootstrap deadlock from the Auto Mode system taint:** the built-in system NodePool's `CriticalAddonsOnly` taint means capacity for an ordinary pod is zero until your own workload NodePool exists — so ArgoCD itself sat Pending and GitOps could not sync at all.
The NodePool and IngressClass manifests were pulled out of the GitOps track and moved into a helmfile pre-bootstrap stage, separating the initial ordering
- **No load balancer, and no error either, from a missing IngressClass:** the IngressClass had been provisioned by the AWS Load Balancer Controller Helm chart, and Auto Mode ships the controller built in without that chart — so nothing created it. Even with `ingressClassName: alb` set there was no matching controller, and the load balancer simply never appeared, with nothing in the logs.
An Auto Mode-specific IngressClass is now its own component, deployed immediately after the cluster is created
- **NodeClass moved API group, and one addon is not included:** a NodePool is still written the same way, but NodeClass moved from `karpenter.k8s.aws/v1` EC2NodeClass to `eks.amazonaws.com/v1` NodeClass, so the existing manifests no longer applied.
The specs were rewritten against the new CRD, and the EFS CSI driver — which Auto Mode does not include — is deployed separately

Results

- Terraform management points in the compute layer went from 102 to 17, an 83% reduction, and node lifecycle and spot reclaim moved entirely to AWS-managed
- Built a site-to-site VPN gateway between on-premises and AWS, keeping the internal ALB off the public internet and preserving a single internal hostname and a single TLS certificate however many environments are added
- To rule out the kind of gray failure where whether the service works depends on where its pod landed, the static route is consolidated at a single point on the gateway router rather than per pod or per node — fail-fast routing, so that if the path disappears everything fails at once
- The three things the tunnel depends on outside Terraform — the router's static route, the gateway config file and the private key — are documented in the runbook, giving operational visibility into dependencies `terraform plan` cannot detect; and the private key is held in AWS SSM Parameter Store as a SecureString so it never sits in plaintext in the tfstate bucket

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the large-scale AWS → GCP cloud migration.

Drove the transition to capture cost optimization and GCP's global network performance, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Maintained dual source management — GitLab for on-premises, GitHub for production — and brought the stalled bare-metal GitLab up the official path in stages (about 12 hops), upgraded in place.

Also built a data-collection pipeline consolidating multi-source data (MongoDB · CloudSQL · GA · Dune) into BigQuery with auto-refreshed Google Sheets, eliminating the 2-3 hours of analysts' daily manual work.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs kept rising, driven largely by per-Pod billing for the always-on game workloads on Fargate.

The migration targeted cost reduction alongside the global load balancing the multi-region game service needed.

Approach

- **Three-phase sequential migration** – Phase 1 built and validated the dev environment on GCP, Phase 2 QA, Phase 3 production
- **Network and authentication** – network designed on Shared VPC (Host Project / Service Project separation); GitHub Actions' GCP auth switched to Workload Identity Federation, replacing service-account key issuance and rotation with short-lived OIDC tokens. DNS pre-validated by subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS), leaving only the final NS swap to cut over inside an announced maintenance window
- **Compute re-platforming** – AWS Fargate workloads moved onto GKE standard node pools; per-Pod billing loses to per-VM billing for always-on workloads, so GKE Autopilot — the same billing model — was ruled out
- **Storage minimum bypassed, WAF** – worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB disk), plus Cloud Armor region blocking and an IP allow-list WAF policy
- **IaC and GitOps pipeline** – codified GCP infrastructure with Terraform (100% excluding IAM), then after the migration built a GitLab CI + ArgoCD GitOps CI/CD pipeline standardizing per-environment config with Helm Charts

Troubleshooting

- While migrating AWS ALB-based routing to a GCP Global LB, traffic-routing issues arose because AWS ALB fails open and still passes traffic when every target is unhealthy, whereas GCP LB blocks it. Analyzed the GCP NEG (Network Endpoint Group) structure and reconfigured health checks/backends for GKE workloads
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response. Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect. Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy
- After delegating DNS to Cloud DNS, the domain was not auto-registered in Google Search Console, breaking search visibility and ownership verification. Added the Search Console verification TXT record to Cloud DNS, verified ownership, and re-registered

Results

- 50% cloud cost reduction — designed and executed the AWS → GCP migration, moved Fargate workloads to GKE standard node pools, and secured an MSP reseller discount
- Completed the full service migration via the 3-phase strategy — minimized downtime with resource-copy migration, and ran the final DNS cutover after announcing an approximately 2-hour maintenance window
- Codified the entire GCP infrastructure as Terraform IaC with 100% deployment history traceable through Git commits
- Eliminated GCP service-account key management overhead and key-leak risk for GitHub Actions by adopting Workload Identity Federation

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily and causing frequent data gaps from human error.

Approach

- **Multi-source ingestion into BigQuery** – MongoDB via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection, Google Analytics · Dune via their own APIs (Dune with a separate API key), all normalized into a consistent index schema
- **Auto-refreshed analyst-facing Google Sheets** – Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, triggered by Cloud Scheduler on Daily (previous-day) and Monthly (previous-month) cadences
- **Full infrastructure as Terraform IaC** – BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM and service accounts all managed as code

Troubleshooting

- The CloudSQL → BigQuery direct connection (federated query) failed on region mismatch and missing connection-SA permissions. Resolved by creating the connection in the same region and granting the connection SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow plus rate limits caused missing results / timeouts. Guaranteed result retrieval via execution-status polling with backoff and moved the API key into Secret Manager
- Daily re-runs loaded duplicate records into BigQuery. Applied MERGE and ROW_NUMBER()-based reconciliation logic in the dedup Cloud Function to make the load idempotent — a re-run

produces the same result — keeping data gaps and duplicates at zero

- Loading BigQuery results into Google Sheets failed on the 10M-cell-per-sheet limit, the write quota, and the per-minute request quota (429).
Worked around the cell limit with chunked values.batchUpdate writes and sheet splitting, and stayed within quota via call pacing/throttling, batching, and exponential backoff

Results

- 95% data collection automation (eliminated 2-3h daily manual work), 0% data gap rate from human error
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

Project 3: GPU Infrastructure Standardization & Scale-out for AI Workloads (Windows → Linux · Automated Setup · On-prem / GCP / External IDC)

Contribution 100% (Infra design, build & operations) 6 months (2024.02 ~ 2024.07)

Problem

The in-house GPU server ran Windows, so engineers hand-configured the runtime for every Stable Diffusion-family image-generation run. Matching driver, CUDA and framework versions by hand made the environment hard to reproduce — the hardware existed, but getting to work took time.

Approach

- **Windows → Linux standardization** - the in-house GPU server moved to Ubuntu 22.04, with a setup script installing and verifying the driver stack (nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6) and nvidia-docker so whoever runs it gets the same environment; the image-generation workloads (Stable Diffusion WebUI · kohya_ss for LoRA training · ComfyUI) sit on top as a standard configuration
- **Cloud scale-out on GCP** - a VM with two NVIDIA A100 40GB cards (`a2-highgpu-2g`) provisioned as Terraform IaC, cleared through per-region/zone A100 availability checks and a GPU quota increase, then attached to the earlier migration's Shared VPC subnet so the network standard held
- **20 more servers at an external IDC, card tier split by workload** - 20 GPU servers built with the same setup script.
Training needs VRAM and a multi-GPU interconnect (NVLink) because the model and batch do not fit on one card, while image-generation inference is independent per request and scales with machine count — so training and large-batch work run on the GCP A100s and bulk inference on consumer RTX 40-series servers at the IDC
- **Hardening alongside the publishers** - a new publishing deal kept the demand coming, and this GPU environment went on being hardened together with the publisher side right up to my departure

Results

- Replaced hand-configuration on Windows with Linux plus a setup script — the same script then provisioned all 20 external-IDC servers, turning a one-box procedure into the standard build path for the whole GPU fleet
- Managed the GPU server as Terraform code rather than a hand-built box, standardizing rebuild and change tracking — folded into the existing GCP IaC repository
- Scaled GPU capacity from a single in-house box to 10 on-prem boxes plus GCP A100 plus 20 servers at an external IDC — card tier split by workload (multi-GPU training vs per-request inference)

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.

Designed and built private cloud infrastructure for financial and public sector clients, and led training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

- **HA cluster design** - Kubernetes clusters designed in HA (High Availability) configuration to match each client's requirements, with a virtual machine management environment built on OpenStack
- **Unified storage** - block, file and object storage provided together through Ceph

Troubleshooting

- **Certificate expiry cycle** - cluster certificates expired on a one-year cycle, carrying a service-outage risk and a renewal job that returned every year at roughly 5 hours per client.

Having analysed the known bug where kubeadm's built-in `certs renew` left some certificates unrenewed on v1.15~1.16, I established a version-independent script ([update-kube-cert](#)) that regenerates the certificates with a ten-year validity and restarts the components, and rolled it out across every client cluster incident-free

Results

- Built and delivered the HA-configured PaaS solution to 5 clients and ran it stably
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and achieving zero configuration drift
- Extended the certificate expiry cycle from one year to ten, eliminating the expiry-outage risk and cutting roughly 50 hours of annual operational effort across the 10 client sites then in operation

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

- **Step-by-step curriculum** - a training curriculum from Kubernetes basics through cluster operations and troubleshooting, run practice-first on hands-on lab environments
- **Parallel PaaS-stack training** - OpenStack and Ceph storage operations taught alongside, securing operational capability across the full PaaS stack

Results

- Ran the curriculum at an average of 10 attendees and 4 hours per session, establishing DP's autonomous Kubernetes operations and reducing client technical inquiries afterwards
- Standardized training materials reused for subsequent client trainings